

COGNIZANT DIGITAL NURTURE 5.0 – DEEP SKILLING PHASE

WEEK 1 : NUnit and Moq – Mandatory Exercises

Submitted By

Name: Dharm Bramhatmaz

Superset ID: 8000524

Email: dbramhatmaz20@gmail.com

University: C. V. Raman Global University, Odisha

Objectives

- Explain the meaning of Unit testing and its difference on comparison with Functional testing
 - Smallest unit to test mocking dependencies
- List various types of testing
 - Unit testing, Functional testing, Automated testing, Performance testing
- Understand the benefit of automated testing
- Explain what is loosely coupled & testable design
 - Write code that is NOT dependent on the class for data.
- Write your first testing program to validate a calculator addition operation
 - TestFixture, Test
- Understand the need of [SetUp], [TearDown] & [Ignore] attributes.
- Explain the benefit of writing parameterised test cases.
 - TestCase

TestFixture & Test

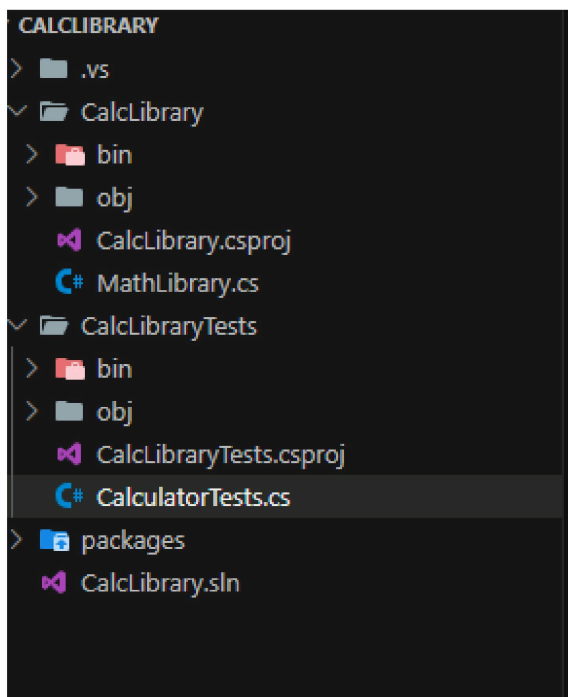
Please download the application available [here](#). This will be used to write Unit test cases

Follow the steps listed below to write the NUnit test cases for the application.

- Create a Unit test project(.Net Framework) in the solution provided.
- Add the CalcLibrary project as reference
- Create a class “CalculatorTests” to write all the test cases for the methods in the solution

- Use the 'TestFixture', 'SetUp' and 'TearDown' attributes, to declare, initialize and cleanup activities respectively
- Create a Test method to check the addition functionality
- Use the 'TestCase' attribute to send the inputs and the expected result
- Use Assert.That to check the actual and expected result match

Project Structure



Filename: CalculatorTests.cs

```
using NUnit.Framework;  
using CalcLibrary;
```

```
namespace CalcLibraryTests  
{
```

```
[TestFixture]
public class CalculatorTests
{
    private SimpleCalculator calculator;

    [SetUp]
    public void SetUp()
    {
        calculator = new SimpleCalculator();
    }

    [TearDown]
    public void TearDown()
    {
        calculator.AllClear();
    }

    [Test]
    public void Addition_Test()
    {
        double result = calculator.Addition(10, 20);

        Assert.That(result, Is.EqualTo(30));
    }

    [TestCase(5, 10, 15)]
    [TestCase(20, 30, 50)]
    [TestCase(-5, 10, 5)]
    [TestCase(100, 200, 300)]
    [TestCase(0, 50, 50)]
}
```

```

public void Addition_TestCases(double a, double b, double expected)
{
    double result = calculator.Addition(a, b);

    Assert.That(result, Is.EqualTo(expected));
}

[Ignore("Ignore Example")]
[Test]
public void Ignore_Test()
{
    Assert.Pass();
}
}
}

```

```

PS C:\Users\user\Downloads\dbramhatmaz20@gmail.com\Dharm_Bramhatmaz\DN_5.0\DOT_NET_FSE\CalcLibrary\CalcLibrary> dotnet test
Restore complete (0.5s)
CalcLibrary netstandard2.0 succeeded (0.1s) -> CalcLibrary\bin\Debug\netstandard2.0\CalcLibrary.dll
CalcLibraryTests net10.0 succeeded (0.1s) -> CalcLibraryTests\bin\Debug\net10.0\CalcLibraryTests.dll
NUnit Adapter 5.0.0.0: Test execution started
Running all tests in C:\Users\user\Downloads\dbramhatmaz20@gmail.com\Dharm_Bramhatmaz\DN_5.0\DOT_NET_FSE\CalcLibrary\CalcLibraryTests\bin\Debug\net10.0\CalcLibraryTests.dll
NUnit3TestExecutor discovered 7 of 7 NUnit test cases using Current Discovery mode, Non-Explicit run
Ignore_Test: Ignore Example
NUnit Adapter 5.0.0.0: Test execution complete
CalcLibraryTests test net10.0 succeeded (1.6s)

Test summary: total: 7, failed: 0, succeeded: 6, skipped: 1, duration: 1.5s
Build succeeded in 2.6s

```